

Chapter 4:

In previous chapters, we explored how JavaServer Pages (JSP) handle the presentation layer using custom tags, JavaBeans, and the Expression Language (EL). However, as web applications grow into enterprise-scale systems, the business logic becomes increasingly complex.

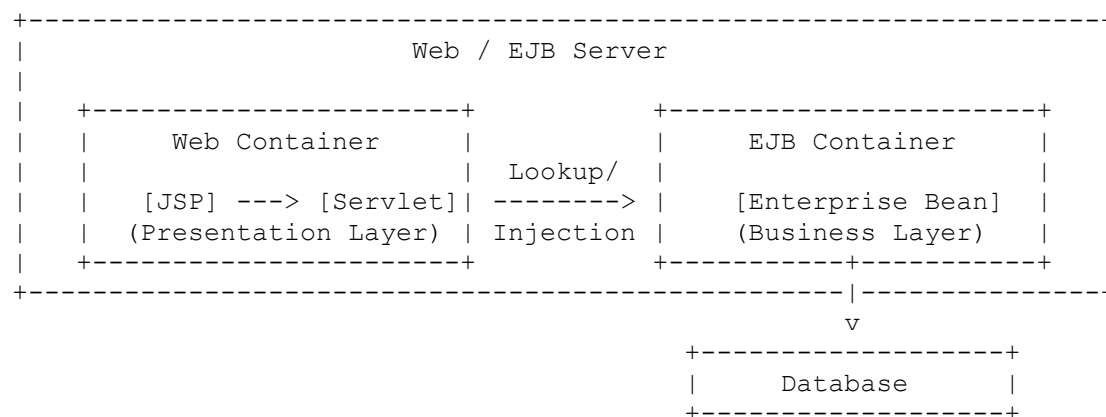
Handling database transactions, security, concurrency, and distributed computing directly within JSPs or standard JavaBeans quickly becomes a maintenance nightmare. This is where **Enterprise Java Beans (EJB)**

1. Introduction to EJB

What is Enterprise Java Beans (EJB)?

Enterprise Java Beans (EJB) is a managed, server-side component architecture for modular development of enterprise applications. It forms the backbone of the Jakarta EE (formerly Java EE) business layer.

While a standard JavaBean runs within the web container and is typically used to hold model data, an EJB runs inside a dedicated **EJB Container** (provided by application servers like WildFly, GlassFish, or WebLogic) that automatically manages complex enterprise services.



Why Use EJB with JSP?

In a standard Model-View-Controller (MVC) architecture using JSP:

- **View:** JSP renders HTML/JSON.
- **Controller:** Servlet handles requests and routing.
- **Model:** EJB encapsulates transactional business logic, security, and data access.

By decoupling business logic into EJBs, your JSP code remains clean, focusing solely on user interface rendering while leveraging robust backend capabilities.

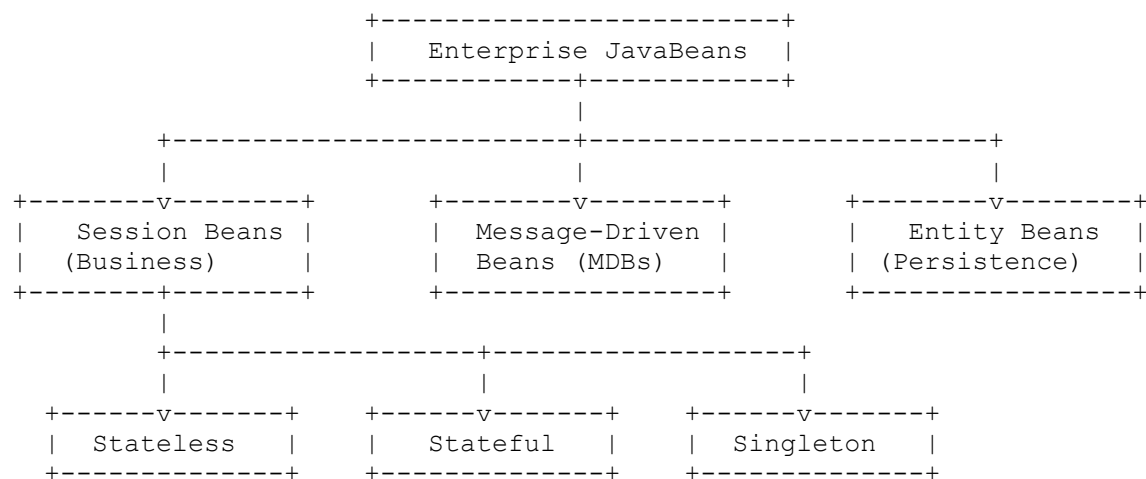
Key Features Provided by the EJB Container

When you write an EJB, the container handles standard "plumbing" tasks out of the box through **Declarative Services**:

- **Transaction Management:** Automatic commit/rollback of multi-step business tasks without writing boilerplate JDBC code.
- **Security:** Declarative, role-based access control (e.g., specifying `@RolesAllowed("ADMIN")`).
- **Concurrency & Lifecycle:** The container automatically handles thread management, instance pooling, and lifecycle hooks.
- **Remote Accessibility:** EJBs can be invoked across networks using remote interfaces.

2. Types of Enterprise Java Beans (EJB)

EJBs are categorized into three distinct types, each designed to solve specific architecture problems:



A. Session Beans

Session beans encapsulate business logic. They execute tasks on behalf of a client (such as a Servlet handling a JSP request).

Session beans are divided into three sub-types based on how they manage client state:

➤ Stateless Session Beans (@Stateless)

- **Behavior:** Do not maintain conversational state with the client. Any instance can service any client request.
- **Container Management:** The container creates a pool of stateless instances. When a request arrives, an instance is picked up from the pool, services the request, and returns to the pool.
- **Use Case:** Operations that are single-step or idempotent, such as fetching a product catalog, sending an email, or calculating tax rates.

Example

```
import jakarta.ejb.Stateless;

@Stateless
public class TaxCalculatorBean {
    public double calculateTax(double amount, double rate) {
        return amount * rate;
    }
}
```

➤ **Stateful Session Beans (@Stateful)**

- **Behavior:** Maintain state across multiple method invocations for a single client session.
- **Container Management:** Dedicated to one client at a time. The container keeps the state in memory and handles passivation (writing to disk if memory is low) and activation.
- **Use Case:** Workflows requiring state retention over multiple requests, such as a multi-step checkout wizard or an e-commerce shopping cart.

Example

```
import jakarta.ejb.Stateful;
import java.util.ArrayList;
import java.util.List;

@Stateful
public class ShoppingCartBean {
    private List<String> items = new ArrayList<>();

    public void addItem(String item) {
        items.add(item);
    }

    public List<String> getItems() {
        return items;
    }
}
```

➤ **Singleton Session Beans (@Singleton)**

- **Behavior:** Instantiated once per application. Shared across all concurrent clients.
- **Container Management:** The container manages concurrent access using automatic locking mechanisms (WRITE or READ locks).
- **Use Case:** Application-wide shared state, such as application configuration settings, in-memory caches, or global hit counters.

Example:

```
import jakarta.ejb.Singleton;
import jakarta.ejb.Startup;
@Singleton
@Startup // Instantiates bean on application startup
public class GlobalConfigBean {
    private String appName = "Enterprise Portal";
    public String getAppName() {
        return appName;
    }
}
```

➤ Message-Driven Beans (MDB) (@MessageDriven)

Message-Driven Beans process requests asynchronously using messaging systems like Java Message Service (JMS).

- **Behavior:** MDBs do not have interfaces exposed to web clients. Instead, they act as asynchronous message listeners.
- **Execution:** When a JMS message arrives at a queue or topic, the container delegates processing to an MDB instance.
- **Use Case:** Background or heavy tasks that shouldn't block user interaction on a JSP page—such as batch processing, generating monthly PDF invoices, or sending push notifications.

Example

```
import jakarta.ejb.MessageDriven;
import jakarta.jms.Message;
import jakarta.jms.MessageListener;

@MessageDriven(mappedName = "jms/OrderQueue")
public class OrderProcessingMDB implements MessageListener {
    @Override
    public void onMessage(Message message) {
        // Asynchronous processing of incoming orders
        System.out.println("Processing async background task...");
    }
}
```

➤ Entity Beans (JPA Entities)

Historically (in EJB 1.x and 2.x), Entity Beans represented persistent database records.

Author Note: Modern Java EE / Jakarta EE development has replaced traditional EJB Entity Beans with the **Java Persistence API (JPA)**. Today, standard Java classes annotated with `@Entity` handle database operations, seamlessly integrating with Session Beans.

Summary Comparison Table

Feature	Stateless Session	Stateful Session	Singleton Session	Message-Driven (MDB)
State Preserved?	No	Yes (per client)	Yes (shared globally)	No
Client Type	Synchronous	Synchronous	Synchronous	Asynchronous (JMS)
Instance Pool	Pooled by container	One instance per client	Single instance	Pooled by container
Typical Usage	Search, Tax calculation	Shopping cart, Wizards	App settings, Cache	Email dispatch, Batch jobs

Introduction to MVC :

In the early days of Java web development (Model 1 Architecture), JSP pages handled both the presentation and the database business logic. This caused messy code that was impossible to maintain.

To fix this, J2EE adopted the **Model 2 Architecture**, which is a server-side implementation of the MVC pattern. It strictly enforces the **Separation of Concerns (SoC)** by distributing roles across standard enterprise Java components.

The J2EE Component Breakdown

- **Model (JavaBeans / EJB / Hibernate POJOs):** Represents the enterprise data structures and handles the core business logic, data validation, and database transactions (JDBC/JPA). It is completely decoupled from the web layer.
- **View (JSP / JSTL / HTML):** The presentation layer. It fetches processed data from request scopes and renders it into a web page. It contains *zero* Java code (no scriptlets) and uses Expression Language (EL) or JSTL tags instead.
- **Controller (HttpServlet):** The central entry point of the web application. It intercepts client requests, parses incoming form or query parameters, invokes the appropriate business Model, and forwards the results to the target View.

❖ The J2EE Request Lifecycle (Data Flow)

When an enterprise application handles an HTTP request, it flows through specific container scopes (HttpServletRequest, HttpSession, ServletContext):

1. **Client Request:** The browser submits an HTTP request (e.g., GET/POST) targeting a servlet mapping (e.g., /userProfile).
2. **Controller Trapping:** The **Servlet (Controller)** receives the request via its doGet() or doPost() methods.
3. **Business Execution:** The Servlet extracts request parameters, instantiates or calls a **JavaBean/Service (Model)**, and processes the business logic.
4. **Scope Storage:** The Model returns data (Java objects or Collections) to the Servlet. The Servlet binds this data to an enterprise scope attribute, most commonly the HttpServletRequest object:
5. request.setAttribute("user", userObj);
6. **Request Dispatching:** The Servlet forwards control programmatically to the **JSP (View)** using a dispatcher:

```
RequestDispatcher rd = request.getRequestDispatcher("profile.jsp");
```
7. rd.forward(request, response);
8. **Presentation Rendering:** The JSP reads the attribute data using Expression Language (`${user.name}`) and outputs pure HTML back to the browser client.

Implementation of MVC:

Below is a production-style implementation tracking a User Profile lookup using standard J2EE APIs.

Step 1: The Model (JavaBean Component)

A standard serializable POJO (Plain Old Java Object) encapsulating data and access rules.

// UserBean.java (The Data Model)

```
package com.faculty.model;
import java.io.Serializable;

public class UserBean implements Serializable {
    private int id;
    private String name;
    private String email;

    // Mandatory No-argument Constructor for JavaBeans
    public UserBean() {}

    public UserBean(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

// UserService.java (The Business Logic Model)

```
package com.faculty.model;

public class UserService {
    public UserBean getUserById(int id) {
        // Simulating a database lookup via JDBC/JPA
        if (id == 101) {
            return new UserBean(101, "Prof. Sharma", "sharma@university.edu");
        }
        return null; // User not found
    }
}
```

```
}
```

Step 2: The Controller (HttpServlet)

Acts as the central router and front-facing traffic controller.

// UserControllerServlet.java (The Controller)

```
package com.faculty.controller;
import com.faculty.model.UserBean;
import com.faculty.model.UserService;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet("/userProfile") // Map URL pattern to this controller
public class UserControllerServlet extends HttpServlet {

    private UserService userService = new UserService();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Extract request parameters
        String idStr = request.getParameter("userId");

        if (idStr != null) {
            int userId = Integer.parseInt(idStr);

            // 2. Interact with Model Layer
            UserBean user = userService.getUserById(userId);

            // 3. Bind Model Data to Request Scope
            request.setAttribute("userData", user);
        }

        // 4. Forward execution to the View (JSP Page)
        request.getRequestDispatcher("/WEB-INF/views/profile.jsp").forward(request,
response);
    }
}
```

Step 3: The View (JSP with Expression Language)

Notice that there is **no raw Java scriptlet** (<% %>) in the presentation file.

```
<!-- profile.jsp (The View placed securely inside WEB-INF) -->
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
```

```

<html>
<head>
  <title>User Profile Dashboard</title>
</head>
<body>
  <h2>Enterprise User Portal</h2>
  <hr/>
  <!-- Accessing Request Attributes safely using J2EE EL -->
  <c:choose>
    <c:when test="\${not empty userData}">
      <table border="1" cellpadding="10">
        <tr>
          <th>User ID</th>
          <td>\${userData.id}</td>
        </tr>
        <tr>
          <th>Full Name</th>
          <td>\${userData.name}</td>
        </tr>
        <tr>
          <th>Email Address</th>
          <td>\${userData.email}</td>
        </tr>
      </table>
    </c:when>
    <c:otherwise>
      <p style="color: red;">Error: Invalid User ID or record not found.</p>
    </c:otherwise>
  </c:choose>
</body>
</html>

```

➤ Introduction to Hibernate

Hibernate is an open-source, lightweight Object-Relational Mapping (ORM) framework for Java. It acts as a bridge between Java applications and relational databases. It dramatically reduces development time by automatically translating Java objects into database rows and generating the necessary SQL queries under the hood.

By handling low-level database operations automatically, Hibernate eliminates the vast majority of tedious JDBC boilerplate code (like establishing connections, manual mapping, and data type conversions).

Database Persistence with Hibernate in JSP

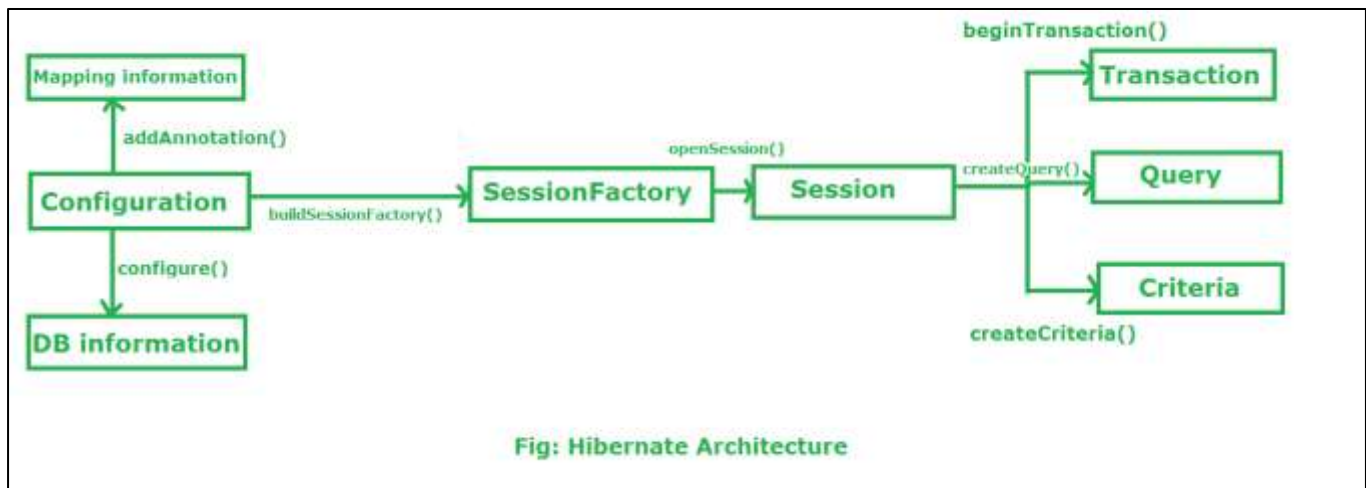
In enterprise applications, bridge mechanisms are needed to connect the object-oriented world of Java with the relational world of SQL databases. While standard JDBC allows JSP and Servlet applications to talk to database tables, it requires writing massive amounts of repetitive

SQL code, manually mapping relational columns to Java objects, and managing database connections.

Feature of Hibernate:

- **Eliminates Boilerplate JDBC Code:** No manual handling of `PreparedStatement`, `ResultSet`, or SQL string concatenations.
- **Database Independence:** Uses abstract query dialects. Switching from MySQL to PostgreSQL requires changing a single configuration property rather than rewriting queries.
- **Caching Mechanisms:** Built-in First-Level (Session) and Second-Level caching reduce database load and improve response times for web requests.
- **Automatic Schema Generation:** Hibernate can auto-generate or update database tables directly from Java entity classes.

Exploring Hibernate Architecture



Configuration

- Configuration is a class which is present in `org.hibernate.cfg` package. It activates Hibernate framework. It reads both configuration file and mapping files.
- It checks whether the config file is syntactically correct or not.
- If the config file is not valid then it will throw an exception. If it is valid then it creates a meta-data in memory and returns the meta-data to object to represent the config file.

It activate Hibernate Framework Configuration `cfg = new Configuration();`
it read both cfg file and mapping files `cfg.configure();`

SessionFactory

- SessionFactory is an Interface which is present in `org.hibernate` package and it is used to create Session Object.
- It is immutable and thread-safe in nature.

buildSessionFactory() method gathers the meta-data which is in the cfg Object. SessionFactory uses the configuration metadata to manage database connections through connection providers and pooling mechanisms.
SessionFactory factory = cfg.buildSessionFactory();

Session

- Session is an interface which is present in org.hibernate package. Session object is created based upon SessionFactory object i.e. factory.
- It opens the Connection/Session with Database software through Hibernate Framework.
- It is a light-weight object and it is not thread-safe.
- Session object is used to perform CRUD operations.

```
Session session = factory.openSession();
```

openSession(): is a method provided by the SessionFactory that creates and returns a new Session instance. This session is not bound to any transaction or context and is independent of any ongoing transactions in the application.

getCurrentSession: it returns a Session bound to the current context, which is usually managed by a transaction manager or a framework like Spring. It requires configuration like hibernate.current_session_context_class (e.g., thread or Spring-managed context).

```
Session session = sessionFactory.getCurrentSession();
```

Transaction

- Transaction object is used whenever we perform any operation and based upon that operation there is some change in database.
- Transaction object is used to give the instruction to the database to make the changes that happen because of operation as a permanent by using commit() method.

```
Transaction tx=session.beginTransaction();  
tx.commit();
```

Query

- Query is an interface in the org.hibernate package used to execute HQL queries.
- It is created using the Session.createQuery() method to interact with the database.
- It supports advanced features like pagination (setMaxResults(), setFirstResult()) and named parameters for better query handling.

Criteria

- Criteria is a simplified API for retrieving entities by composing Criterion objects.
- The Session is a factory for Criteria. Criterion instances are usually obtained via the factory methods on Restrictions.

To eliminate this friction, Java developers use **Object-Relational Mapping (ORM)** framework with **Hibernate** being the industry standard. Here with explores how Hibernate simplifies persistence management in web applications.

➤ Object-Relational Mapping (ORM) with Hibernate

What is ORM?

Object-Relational Mapping (ORM) is a programming technique that automatically maps Java classes to database tables, and Java data types to SQL data types.

Java Application	Hibernate	Relational Database
User (Java Class)	<=====>	users (SQL Table)
- id: Long	(ORM)	- id: BIGINT (PK)
- name: String		- name: VARCHAR
- email: String		- email: VARCHAR

Without ORM, developers must write manual CRUD (Create, Read, Update, Delete) operations using JDBC result sets. With ORM, you interact directly with Java objects, and the framework generates and executes the appropriate SQL statements behind the scenes.

➤ The Hibernate Configuration File (hibernate.cfg.xml)

The `hibernate.cfg.xml` file is the central nervous system of a Hibernate application. It defines database connections, driver details, connection pooling, dialect selections, and references to mapped entities.

Place this file under the application's root classpath (`src/main/resources` or `WEB-INF/classes`).

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <!-- 1. Database Connection Settings -->
        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property
>

        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/jsp_db?useSSL=f
alse</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">secret123</property>

        <!-- 2. SQL Dialect & Developer Tools -->
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>

        <!-- 3. Schema Auto-Creation Policy (create, update, validate) -->
```

```

        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- 4. Registered Mapped Entities -->
        <mapping class="com.example.model.Student"/>
    </session-factory>
</hibernate-configuration>

```

➤ **Hibernate Mapping File (.hbm.xml)**

Historically, mapping between a Java object and a database table was defined using XML files (.hbm.xml). While JPA Annotations have largely superseded XML files, understanding XML mapping is crucial for legacy codebase maintenance.

Suppose we have a standard Java Bean class, `Student.java`:

Java

```

package com.example.model;

public class Student {
    private int id;
    private String name;
    private String email;

    // Getters and Setters omitted for brevity
}

```

The corresponding **`Student.hbm.xml`** mapping file maps this Java class to a database table:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping package="com.example.model">
    <class name="Student" table="students">
        <!-- Primary Key Mapping -->
        <id name="id" column="student_id">
            <generator class="native"/> <!-- Auto-increment strategy -->
        </id>

        <!-- Column Properties -->
        <property name="name" column="student_name" type="string"/>
        <property name="email" column="student_email" type="string"/>
    </class>
</hibernate-mapping>

```

➤ **Hibernate Annotations**

Modern enterprise applications prefer **JPA/Hibernate Annotations** over XML files. Annotations place database metadata directly inside the Java class, consolidating class definitions and mapping metadata into a single source file.

Here is Student.java rewritten using annotations:

```
package com.example.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;

@Entity                                // Marks class as a persistent
entity
@Table(name = "students")              // Maps to SQL table 'students'
public class Student {

    @Id                                // Marks primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increment
    ID
    @Column(name = "student_id")
    private int id;

    @Column(name = "student_name", nullable = false)
    private String name;

    @Column(name = "student_email", unique = true)
    private String email;

    // Default No-Arg Constructor (Required by Hibernate)
    public Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Essential Annotations Reference

Annotation	Purpose
<code>@Entity</code>	Declares a Java class as an ORM managed domain model.
<code>@Table (name="...")</code>	Specifies the exact database table name.
<code>@Id</code>	Marks a field as the Primary Key of the table.
<code>@GeneratedValue</code>	Defines Primary Key auto-generation strategies (<code>IDENTITY</code> , <code>SEQUENCE</code> , <code>AUTO</code>).
<code>@Column (name="...")</code>	Maps a Java class field to a specific database table column name.
<code>@Transient</code>	Instructs Hibernate to ignore a field (does not store it in DB).

5. Hibernate Query Language (HQL)

Hibernate Query Language (HQL) is an object-oriented extension of SQL. Instead of querying database tables and columns, HQL queries Java **Entities** and **Properties**.

SQL vs. HQL Key Differences

- **SQL:** `SELECT * FROM students WHERE student_name = 'Alice';` (*Operates on tables and columns*)
- **HQL:** `FROM Student s WHERE s.name = 'Alice'` (*Operates on class names and property variables*)

HQL Usage Examples

1. Fetching All Records

Java

```
String hql = "FROM Student";
Query<Student> query = session.createQuery(hql, Student.class);
List<Student> students = query.getResultList();
```

2. Parameterized HQL Query (Prevents SQL Injection)

Java

```
String hql = "FROM Student s WHERE s.email = :paramEmail";
Query<Student> query = session.createQuery(hql, Student.class);
query.setParameter("paramEmail", "john@example.com");

Student student = query.getSingleResult();
```

3. HQL Update & Delete

Java

```
// Update Operation
Transaction tx = session.beginTransaction();
String updateHql = "UPDATE Student s SET s.email = :newEmail WHERE s.id = :studentId";
Query query = session.createQuery(updateHql);
query.setParameter("newEmail", "updated@example.com");
query.setParameter("studentId", 1);
int rowsAffected = query.executeUpdate();
tx.commit();
```

➤ Hibernate Sessions and Lifecycle Management

To interact with the database using Hibernate, you need to understand two key concepts: the **SessionFactory** and the **Session**.

```
SessionFactory (Heavyweight, Thread-Safe, Created once on Startup)
    |
    v
Session (Lightweight, Non Thread-Safe, Per Web Request)
    |
    v
Transaction (Groups DB Operations Together)
```

1. **SessionFactory**: A heavy, thread-safe object initialized once during application startup. It compiles configuration metadata and acts as a factory for opening database connections.
2. **Session**: A short-lived, lightweight object wrapping a single JDBC connection. Represents a physical unit-of-work with the database. **Not thread-safe**.

Utility Class to Manage SessionFactory

```
package com.example.util;
```

```

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
    private static final SessionFactory sessionFactory =
        buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            // Loads hibernate.cfg.xml automatically
            return new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation failed: " +
ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}

```

Complete End-to-End Example: Fetching Data inside a JSP Context

Servlet (Controller Layer)

Java

```

@WebServlet("/listStudents")
public class StudentServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        // Open Hibernate Session
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            // Execute HQL Query
            List<Student> studentList = session.createQuery("FROM Student",
Student.class).list();

            // Set Attribute and Forward to JSP
            request.setAttribute("students", studentList);

            request.getRequestDispatcher("displayStudents.jsp").forward(request,
response);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

JSP View Layer (displayStudents.jsp)

Java

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

```

```
<html>
<head>
  <title>Student Roster</title>
</head>
<body>
  <h2>Registered Students List</h2>
  <table border="1" cellpadding="8">
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
    </tr>
    <c:forEach var="student" items="${students}">
      <tr>
        <td>${student.id}</td>
        <td><c:out value="${student.name}"/></td>
        <td><c:out value="${student.email}"/></td>
      </tr>
    </c:forEach>
  </table>
</body>
</html>
```